

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

*I **R.Pranathi(192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

ANS:

Given Relation

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Understanding attributes

Attribute	Meaning
OrderID	Unique order number
CustID	Customer ID
CustName	Customer name
ProdID	Product ID
ProdName	Product name
Qty	Quantity ordered
Price	Price of the product

Assume that one order can contain multiple products.

Example:

OrderID	CustID	CustName	ProdID	ProdName	Qty	Price
O101	C01	Ravi	P01	Laptop	1	50000
O101	C01	Ravi	P02	Mouse	2	1000
O102	C02	Anu	P01	Laptop	1	50000

Primary Key

Because an order can contain multiple products, OrderID alone is not sufficient.
Therefore:

Primary Key = (OrderID, ProdID)

It is a composite primary key.

Functional Dependencies

From the meaning of the attributes:

FD 1 – Customer

A customer ID determines the customer's name:

$CustID \rightarrow CustName$

FD 2 – Product

A product ID determines the product name and price:

$ProdID \rightarrow ProdName, Price$

FD 3 – Order Item

The combination of order and product determines the quantity:

$(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$

Therefore, write all FDs as:

$\text{CustID} \rightarrow \text{CustName}$

$\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$

$(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$

Check 1NF

A table is in 1NF when every attribute contains atomic/single values.

Our relation contains single values in every cell.

Therefore:

ORDER is in 1NF.

So we have:

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Check 2NF

The primary key is:

$(\text{OrderID}, \text{ProdID})$

Now check whether any non-key attribute depends on only part of this composite key.

We have:

$\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$

Convert to 2NF

Create three tables.

1. CUSTOMER

CUSTOMER(CustID, CustName)

Primary key:

CustID

2. PRODUCT

PRODUCT(ProdID, ProdName, Price)

Primary key:

ProdID

3. ORDER_DETAILS

ORDER_DETAILS(OrderID, CustID, ProdID, Qty)

Primary key:

$(\text{OrderID}, \text{ProdID})$

Foreign keys:

$\text{CustID} \rightarrow \text{CUSTOMER}(\text{CustID})$

$\text{ProdID} \rightarrow \text{PRODUCT}(\text{ProdID})$

Check 3NF

CUSTOMER

$\text{CustID} \rightarrow \text{CustName}$

CustID is the primary key.

Therefore, CUSTOMER is in 3NF.

PRODUCT

$\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$

ProdID is the primary key.

Therefore, PRODUCT is in 3NF.

ORDER_DETAILS

(OrderID, ProdID) $\rightarrow$ Qty

The determinant is the primary key.

Therefore, ORDER_DETAILS is in 3NF.

Final 3NF Design

CUSTOMER

CustID PK

CustName

PRODUCT

ProdID PK

ProdName

Price

ORDER_DETAILS

OrderID PK

CustID FK

ProdID FK

Qty

PK = (OrderID, ProdID)

Relationship:

CUSTOMER

|

| 1 : M

↓

ORDER_DETAILS

↑

| M : 1

|

PRODUCT

```
mysql> CREATE DATABASE OnlineShopping;
Query OK, 1 row affected (0.05 sec)

mysql> use onlineshopping;
Database changed
mysql> CREATE TABLE Customer ( CustID VARCHAR(10) PRIMARY KEY, CustName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.29 sec)

mysql> CREATE TABLE Product ( ProdID VARCHAR(10) PRIMARY KEY, ProdName VARCHAR(50) NOT NULL, Price DECIMAL(10,2) NOT NULL );
Query OK, 0 rows affected (0.13 sec)

mysql> CREATE TABLE Order_Details ( OrderID VARCHAR(10), CustID VARCHAR(10), ProdID VARCHAR(10), Qty INT NOT NULL, PRIMARY KEY (OrderID, ProdID), FOREIGN KEY (CustID) REFERENCES Customer(CustID), FOREIGN KEY (ProdID) REFERENCES Product(ProdID) );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Customer VALUES ('C01', 'sonu'), ('C02', 'pranathi'), ('C03', 'roops');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Product VALUES ('P01', 'Laptop', 50000), ('P02', 'Mouse', 1000), ('P03', 'Keyboard', 2000);
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Order_Details VALUES ('O101', 'C01', 'P01', 1), ('O101', 'C01', 'P02', 2), ('O102', 'C02', 'P01', 1), ('O103', 'C03', 'P03', 1);
Query OK, 4 rows affected (0.09 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Customer;
+-----+-----+
| CustID | CustName |
+-----+-----+
| C01    | sonu     |
| C02    | pranathi |
| C03    | roops    |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProdID | ProdName | Price |
+-----+-----+-----+
| P01    | Laptop   | 50000.00 |
| P02    | Mouse    | 1000.00  |
| P03    | Keyboard | 2000.00  |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProdID | ProdName | Price |
+-----+-----+-----+
| P01    | Laptop   | 50000.00 |
| P02    | Mouse    | 1000.00  |
| P03    | Keyboard | 2000.00  |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Order_Details;
+-----+-----+-----+-----+
| OrderID | CustID | ProdID | Qty |
+-----+-----+-----+-----+
| O101    | C01    | P01    | 1   |
| O101    | C01    | P02    | 2   |
| O102    | C02    | P01    | 1   |
| O103    | C03    | P03    | 1   |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT o.OrderID, c.CustID, c.CustName, p.ProdID, p.ProdName, o.Qty, p.Price FROM Order_Details o JOIN Customer c ON o.CustID = c.CustID JOIN Product p ON o.ProdID = p.ProdID;
+-----+-----+-----+-----+-----+-----+-----+
| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
+-----+-----+-----+-----+-----+-----+-----+
| O101    | C01    | sonu     | P01    | Laptop   | 1   | 50000.00 |
| O101    | C01    | sonu     | P02    | Mouse    | 2   | 1000.00  |
| O102    | C02    | pranathi | P01    | Laptop   | 1   | 50000.00 |
| O103    | C03    | roops    | P03    | Keyboard | 1   | 2000.00  |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Result:

The given ORDER relation has the functional dependencies $CustID \rightarrow CustName$, $ProdID \rightarrow ProdName, Price$, and $(OrderID, ProdID) \rightarrow Qty$. The composite key is $(OrderID, ProdID)$. The relation is initially in 1NF but contains partial dependencies, so it is decomposed into CUSTOMER, PRODUCT, and ORDER_DETAILS. The resulting relations are in 3NF because every non-key attribute depends on the key, the whole key, and nothing but the key. The decomposition reduces redundancy and prevents insertion, update, and deletion anomalies.

2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

ANS:

Given Relation

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Primary Key

PK = PID

Functional Dependencies

$PID \rightarrow PName, DocID, WardNo$

$DocID \rightarrow DocName, Specialization$

$WardNo \rightarrow WardName$

Anomalies

- Update: Doctor/ward details must be updated in multiple rows.
- Insertion: A new doctor cannot be stored without a patient.
- Deletion: Deleting the last patient may delete doctor/ward information.

1NF

All attributes contain atomic values.

✓ 1NF

2NF

PID is a single-attribute key, so there are no partial dependencies.

✓ 2NF

3NF Check

Transitive dependencies exist:

$PID \rightarrow DocID \rightarrow DocName, Specialization$

$PID \rightarrow WardNo \rightarrow WardName$

Therefore:

✗ Not in 3NF

Decompose to BCNF

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

Keys:

PATIENT $\rightarrow$ PID

DOCTOR $\rightarrow$ DocID

WARD $\rightarrow$ WardNo

BCNF Verification

$PID \rightarrow PName, DocID, WardNo$

$DocID \rightarrow DocName, Specialization$

$WardNo \rightarrow WardName$

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE HospitalDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE HospitalDB;
Database changed
mysql> CREATE TABLE Doctor ( DocID VARCHAR(10) PRIMARY KEY, DocName VARCHAR(50) NOT NULL, Specialization VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Ward ( WardNo VARCHAR(10) PRIMARY KEY, WardName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.11 sec)

mysql> CREATE TABLE Patient ( PID VARCHAR(10) PRIMARY KEY, PName VARCHAR(50) NOT NULL, DocID VARCHAR(10), WardNo VARCHAR(10), FOREIGN KEY (DocID) REFERENCES
Doctor(DocID), FOREIGN KEY (WardNo) REFERENCES Ward(WardNo) );
Query OK, 0 rows affected (0.16 sec)

mysql> INSERT INTO Doctor VALUES ('D01', 'Dr. Sonu', 'Cardiology'), ('D02', 'Dr. Pranathi', 'Neurology'), ('D03', 'Dr. Roopa', 'Orthopedics');
Query OK, 3 rows affected (0.03 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Ward VALUES ('W01', 'Cardiology Ward'), ('W02', 'Neurology Ward'), ('W03', 'Orthopedic Ward');
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Patient VALUES ('P01', 'Abhighna', 'D01', 'W01'), ('P02', 'Aishu', 'D01', 'W01'), ('P03', 'Krishna', 'D02', 'W02'), ('P04', 'Sunitha', 'D
02', 'W02');
Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> select*from doctor;
+-----+-----+-----+
| DocID | DocName | Specialization |
+-----+-----+-----+
| D01   | Dr. Sonu | Cardiology     |
| D02   | Dr. Pranathi | Neurology     |
| D03   | Dr. Roopa | Orthopedics    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select*from doctor;
+-----+-----+-----+
| DocID | DocName | Specialization |
+-----+-----+-----+
| D01   | Dr. Sonu | Cardiology     |
| D02   | Dr. Pranathi | Neurology     |
| D03   | Dr. Roopa | Orthopedics    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select*from ward;
+-----+-----+
| WardNo | WardName |
+-----+-----+
| W01    | Cardiology Ward |
| W02    | Neurology Ward |
| W03    | Orthopedic Ward |
+-----+-----+
3 rows in set (0.00 sec)

mysql> select*from patient;
+-----+-----+-----+-----+
| PID | PName | DocID | WardNo |
+-----+-----+-----+-----+
| P01 | Abhighna | D01 | W01 |
| P02 | Aishu | D01 | W01 |
| P03 | Krishna | D02 | W02 |
| P04 | Sunitha | D02 | W02 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT p.PID, p.PName, d.DocID, d.DocName, d.Specialization, w.WardNo, w.WardName FROM Patient p JOIN Doctor d ON p.DocID = d.DocID JOIN Ward w ON p.
WardNo = w.WardNo;
+-----+-----+-----+-----+-----+-----+-----+
| PID | PName | DocID | DocName | Specialization | WardNo | WardName |
+-----+-----+-----+-----+-----+-----+-----+
| P01 | Abhighna | D01 | Dr. Sonu | Cardiology | W01 | Cardiology Ward |
| P02 | Aishu | D01 | Dr. Sonu | Cardiology | W01 | Cardiology Ward |
| P03 | Krishna | D02 | Dr. Pranathi | Neurology | W02 | Neurology Ward |
| P04 | Sunitha | D02 | Dr. Pranathi | Neurology | W02 | Neurology Ward |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Result:

The original relation has update, insertion and deletion anomalies due to redundancy. Decomposing it into PATIENT, DOCTOR and WARD removes transitive dependencies and produces a BCNF design.

3. A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

ANS:

Given Relation

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

Functional Dependencies

SID → SName

CourseID → CourseName, Faculty

(SID, CourseID) → Grade

Primary Key

Since a student can enroll in multiple courses:

Primary Key = (SID, CourseID)

It is a composite key.

1NF Analysis

The relation is in 1NF because all attributes contain atomic values and there are no repeating groups.

✓ 1NF

Identify Partial Dependencies

The composite key is:

(SID, CourseID)

But:

$SID \rightarrow SName$

SName depends only on SID.

Also:

$CourseID \rightarrow CourseName, Faculty$

CourseName and Faculty depend only on CourseID.

Therefore, partial dependencies exist.

X Not in 2NF

2NF Decomposition

STUDENT

STUDENT(SID, SName)

Primary Key: SID

COURSE

COURSE(CourseID, CourseName, Faculty)

Primary Key: CourseID

ENROLLMENT

ENROLLMENT(SID, CourseID, Grade)

Primary Key: (SID, CourseID)

Foreign Keys:

$SID \rightarrow STUDENT(SID)$

$CourseID \rightarrow COURSE(CourseID)$

Final 2NF Schema

STUDENT

SID (PK)

SName

COURSE

CourseID (PK)

CourseName

Faculty

ENROLLMENT

SID (FK)

CourseID (FK)

Grade

PK = (SID, CourseID)

```
mysql> CREATE DATABASE UniversityDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE UniversityDB;
Database changed
mysql> CREATE TABLE Student ( SID VARCHAR(10) PRIMARY KEY, SName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.14 sec)

mysql> CREATE TABLE Course ( CourseID VARCHAR(10) PRIMARY KEY, CourseName VARCHAR(50) NOT NULL, Faculty VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.15 sec)

mysql> CREATE TABLE Enrollment ( SID VARCHAR(10), CourseID VARCHAR(10), Grade VARCHAR(5), PRIMARY KEY (SID, CourseID), FOREIGN KEY (SID) REFERENCES Student(SID), FOREIGN KEY (CourseID) REFERENCES Course(CourseID) );
Query OK, 0 rows affected (0.18 sec)

mysql> INSERT INTO Student VALUES ('S01', 'ravuru'), ('S02', 'aishu'), ('S03', 'krishna');
Query OK, 3 rows affected (0.04 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Course VALUES ('C01', 'DBMS', 'Dr. Kiran'), ('C02', 'Operating Systems', 'Dr. Pranathi'), ('C03', 'Python', 'Dr. Ramesh');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Enrollment VALUES ('S01', 'C01', 'A'), ('S01', 'C02', 'B+'), ('S02', 'C01', 'A+'), ('S03', 'C03', 'A');
Query OK, 4 rows affected (0.06 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+
| SID | SName |
+-----+-----+
| S01 | ravuru |
| S02 | aishu |
| S03 | krishna |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Course;
+-----+-----+-----+
| CourseID | CourseName | Faculty |
+-----+-----+-----+
| C01 | DBMS | Dr. Kiran |
| C02 | Operating Systems | Dr. Pranathi |
+-----+-----+-----+
```

```
mysql> SELECT * FROM Course;
+-----+-----+-----+
| CourseID | CourseName | Faculty |
+-----+-----+-----+
| C01 | DBMS | Dr. Kiran |
| C02 | Operating Systems | Dr. Pranathi |
| C03 | Python | Dr. Ramesh |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| SID | CourseID | Grade |
+-----+-----+-----+
| S01 | C01 | A |
| S01 | C02 | B+ |
| S02 | C01 | A+ |
| S03 | C03 | A |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT e.SID, s.SName, e.CourseID, c.CourseName, c.Faculty, e.Grade FROM Enrollment e JOIN Student s ON e.SID = s.SID JOIN Course c ON e.CourseID = c.CourseID;
+-----+-----+-----+-----+-----+-----+
| SID | SName | CourseID | CourseName | Faculty | Grade |
+-----+-----+-----+-----+-----+-----+
| S01 | ravuru | C01 | DBMS | Dr. Kiran | A |
| S01 | ravuru | C02 | Operating Systems | Dr. Pranathi | B+ |
| S02 | aishu | C01 | DBMS | Dr. Kiran | A+ |
| S03 | krishna | C03 | Python | Dr. Ramesh | A |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Result:

The ENROLLMENT relation has partial dependencies because SName depends only on SID, while CourseName and Faculty depend only on CourseID. Therefore, the relation is not in 2NF. It is decomposed into STUDENT, COURSE, and ENROLLMENT relations, removing the partial dependencies and achieving 2NF.

4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

ANS:

Given Relation

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Primary Key

Assume one passenger can have one seat on a particular flight:

Primary Key = (FlightNo, PassID)

Functional Dependencies

$\text{PassID} \rightarrow \text{PassName}$

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

$(\text{FlightNo}, \text{PassID}) \rightarrow \text{Seat}, \text{Date}$

1NF

All attributes contain atomic values.

✓ 1NF

Check 2NF

The primary key is:

(FlightNo, PassID)

Partial dependencies exist:

$\text{PassID} \rightarrow \text{PassName}$

and

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

Therefore:

✗ Not in 2NF

Decompose into 2NF

PASSENGER

PASSENGER(PassID, PassName)

PK = PassID

FLIGHT

FLIGHT(FlightNo, DeptCity, ArrCity)

PK = FlightNo

BOOKING

BOOKING(FlightNo, PassID, Seat, Date)

PK = (FlightNo, PassID)

Check 3NF

In the decomposed tables:

$\text{PassID} \rightarrow \text{PassName}$

PassID is a key in PASSENGER.

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

FlightNo is a key in FLIGHT.

$(\text{FlightNo}, \text{PassID}) \rightarrow \text{Seat}, \text{Date}$

(FlightNo, PassID) is a key in BOOKING.

Therefore:

✓ 3NF

Check BCNF

BCNF requires that every determinant must be a superkey.

PASSENGER

$\text{PassID} \rightarrow \text{PassName}$

PassID is the primary key.

✓ BCNF

FLIGHT

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

FlightNo is the primary key.

✓ BCNF

BOOKING

$(\text{FlightNo}, \text{PassID}) \rightarrow \text{Seat}, \text{Date}$

(FlightNo, PassID) is the primary key.

✓ BCNF

Therefore, the final decomposition is in BCNF.

Final BCNF Schema

PASSENGER

PassID (PK)

PassName

FLIGHT

FlightNo (PK)

DeptCity

ArrCity

BOOKING

FlightNo (FK)

PassID (FK)

Seat

Date

PK = (FlightNo, PassID)

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE AirlineDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE AirlineDB;
Database changed
mysql> CREATE TABLE Passenger ( PassID VARCHAR(10) PRIMARY KEY, PassName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Flight ( FlightNo VARCHAR(10) PRIMARY KEY, DeptCity VARCHAR(50), ArrCity VARCHAR(50) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Booking ( FlightNo VARCHAR(10), PassID VARCHAR(10), Seat VARCHAR(10), Date DATE, PRIMARY KEY (FlightNo, PassID), FOREIGN KEY (FlightNo)
REFERENCES Flight(FlightNo), FOREIGN KEY (PassID) REFERENCES Passenger(PassID) );
Query OK, 0 rows affected (0.20 sec)

mysql> INSERT INTO Passenger VALUES ('P01', 'Ramesh'), ('P02', 'Aishu'), ('P03', 'Pranathi');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Flight VALUES ('F101', 'Chennai', 'Delhi'), ('F102', 'Mumbai', 'Bangalore');
Query OK, 2 rows affected (0.07 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Booking VALUES ('F101', 'P01', '12A', '2026-08-15'), ('F101', 'P02', '12B', '2026-08-15'), ('F102', 'P03', '15C', '2026-08-16');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT b.FlightNo, p.PassID, p.PassName, b.Seat, f.DeptCity, f.ArrCity, b.Date FROM Booking b JOIN Passenger p ON b.PassID = p.PassID JOIN Flight f ON
b.FlightNo = f.FlightNo;
+-----+-----+-----+-----+-----+-----+
| FlightNo | PassID | PassName | Seat | DeptCity | ArrCity | Date |
+-----+-----+-----+-----+-----+-----+
| F101    | P01    | Ramesh   | 12A  | Chennai  | Delhi   | 2026-08-15 |
| F101    | P02    | Aishu    | 12B  | Chennai  | Delhi   | 2026-08-15 |
| F102    | P03    | Pranathi | 15C  | Mumbai   | Bangalore | 2026-08-16 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)
```

Result:

The original BOOKING relation contains partial dependencies because PassID determines PassName and FlightNo determines DeptCity and ArrCity. The relation is decomposed into PASSENGER, FLIGHT, and BOOKING. In each resulting relation, every determinant is a superkey. Therefore, the final design satisfies BCNF.

5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

ANS:

Relation

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Primary Key

EmpID

Functional Dependencies

EmpID → EmpName, DeptID, ManagerID

DeptID → DeptName

Transitive Dependency

EmpID → DeptID

DeptID → DeptName

Therefore:

EmpID → DeptName

So DeptName is transitively dependent on EmpID.

3NF Decomposition

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID)

DEPARTMENT(DeptID, DeptName)

Keys

EMPLOYEE

PK = EmpID

FK = DeptID

DEPARTMENT

PK = DeptID

```
mysql> CREATE TABLE Employee ( EmpID VARCHAR(10) PRIMARY KEY, EmpName VARCHAR(50) NOT NULL, DeptID VARCHAR(10), ManagerID VARCHAR(10), FOREIGN KEY (DeptID)
REFERENCES Department(DeptID) );
Query OK, 0 rows affected (0.16 sec)

mysql> INSERT INTO Department VALUES ('D01', 'IT'), ('D02', 'HR'), ('D03', 'Finance');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Employee VALUES ('E01', 'Ramesh', 'D01', 'E05'), ('E02', 'Aishu', 'D02', 'E06'), ('E03', 'Sunitha', 'D03', 'E07');
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| D01    | IT       |
| D02    | HR       |
| D03    | Finance  |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | ManagerID |
+-----+-----+-----+-----+
| E01   | Ramesh  | D01    | E05       |
| E02   | Aishu   | D02    | E06       |
| E03   | Sunitha | D03    | E07       |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT e.EmpID, e.EmpName, e.DeptID, d.DeptName, e.ManagerID FROM Employee e JOIN Department d ON e.DeptID = d.DeptID;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| E01   | Ramesh  | D01    | IT       | E05       |
| E02   | Aishu   | D02    | HR       | E06       |
| E03   | Sunitha | D03    | Finance  | E07       |
+-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)
```

6. Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

ANS:

Given Relation

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Functional Dependencies

MemberID → MemberName

BookID → Title, Author, Genre

Identify MVD

If a member can borrow multiple books independently:

MemberID →→ BookID

This means one MemberID can be associated with multiple BookID values independently.

Check 4NF

The relation contains a non-trivial MVD:

MemberID →→ BookID

but MemberID is not a superkey of the complete LIBRARY relation.

Therefore:

✗ LIBRARY is not in 4NF

4NF Decomposition

MEMBER

MEMBER(MemberID, MemberName)

PK = MemberID

BOOK

BOOK(BookID, Title, Author, Genre)

PK = BookID

BORROWING

BORROWING(MemberID, BookID, BorrowDate)

PK = (MemberID, BookID)

4NF Schema

MEMBER

MemberID (PK)

MemberName

BOOK

BookID (PK)

Title

Author

Genre

BORROWING

MemberID (FK)

BookID (FK)

BorrowDate

PK = (MemberID, BookID)

```
mysql> CREATE DATABASE LibraryDB;
Query OK, 1 row affected (0.06 sec)

mysql> USE LibraryDB;
Database changed
mysql> CREATE TABLE Member ( MemberID VARCHAR(10) PRIMARY KEY, MemberName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Book ( BookID VARCHAR(10) PRIMARY KEY, Title VARCHAR(100) NOT NULL, Author VARCHAR(50), Genre VARCHAR(50) );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Borrowing ( MemberID VARCHAR(10), BookID VARCHAR(10), BorrowDate DATE, PRIMARY KEY (MemberID, BookID), FOREIGN KEY (MemberID) REFERENCES Member(MemberID), FOREIGN KEY (BookID) REFERENCES Book(BookID) );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Member VALUES ('M01', 'Ramesh'), ('M02', 'Sunitha'), ('M03', 'Lehan');
Query OK, 3 rows affected (0.06 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Book VALUES ('B01', 'DBMS', 'Kumar', 'Education'), ('B02', 'Operating Systems', 'Galvin', 'Education'), ('B03', 'Python', 'Mark Lutz', 'Programming');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Borrowing VALUES ('M01', 'B01', '2026-08-01'), ('M01', 'B02', '2026-08-03'), ('M02', 'B03', '2026-08-05');
Query OK, 3 rows affected (0.07 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT m.MemberID, m.MemberName, b.BookID, b.Title, b.Author, b.Genre, br.BorrowDate FROM Borrowing br JOIN Member m ON br.MemberID = m.MemberID JOIN Book b ON br.BookID = b.BookID;
+-----+-----+-----+-----+-----+-----+-----+
| MemberID | MemberName | BookID | Title           | Author   | Genre   | BorrowDate |
+-----+-----+-----+-----+-----+-----+-----+
| M01      | Ramesh     | B01    | DBMS            | Kumar    | Education | 2026-08-01 |
| M01      | Ramesh     | B02    | Operating Systems | Galvin   | Education | 2026-08-03 |
| M02      | Sunitha    | B03    | Python          | Mark Lutz | Programming | 2026-08-05 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.05 sec)
```

Result:

The MVD MemberID →→ BookID causes redundancy in the original relation. The relation is decomposed into MEMBER, BOOK, and BORROWING. The resulting relations eliminate the multivalued dependency and satisfy 4NF.

7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization

ANS:

Candidate Keys

A candidate key is a minimal attribute or set of attributes that uniquely identifies each record.

Candidate Keys = To be identified from the given telecom schema

Primary Key

The primary key is one candidate key selected to uniquely identify each record.

Primary Key = One of the candidate keys

Functional Dependencies

A functional dependency (FD) shows that one attribute determines another.

Write them in the form:

$$A \rightarrow B$$

For example, if CustomerID uniquely identifies a customer:

$$\text{CustomerID} \rightarrow \text{CustomerName, PhoneNo}$$

If BillID uniquely identifies a bill:

$$\text{BillID} \rightarrow \text{CustomerID, BillDate, Amount}$$

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE TelecomDB;
Query OK, 1 row affected (0.05 sec)

mysql> USE TelecomDB;
Database changed
mysql> CREATE TABLE Telecom ( CustomerID VARCHAR(10), CustomerName VARCHAR(50), PhoneNo VARCHAR(15), BillID VARCHAR(10), BillDate DATE, Amount DECIMAL(10,2)
, PRIMARY KEY (BillID), UNIQUE (PhoneNo), UNIQUE (CustomerID) );
Query OK, 0 rows affected (0.17 sec)

mysql> INSERT INTO Telecom VALUES ('C01', 'Sonu', '9876543210', 'B01', '2026-08-01', 599.00), ('C02', 'Pranathi', '9876543211', 'B02', '2026-08-02', 799.00)
('C03', 'Roopa', '9876543212', 'B03', '2026-08-03', 499.00);
Query OK, 3 rows affected (0.20 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Telecom;
+-----+-----+-----+-----+-----+-----+
| CustomerID | CustomerName | PhoneNo | BillID | BillDate | Amount |
+-----+-----+-----+-----+-----+-----+
| C01       | Sonu        | 9876543210 | B01    | 2026-08-01 | 599.00 |
| C02       | Pranathi    | 9876543211 | B02    | 2026-08-02 | 799.00 |
| C03       | Roopa       | 9876543212 | B03    | 2026-08-03 | 499.00 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.04 sec)

mysql> SELECT CustomerID, COUNT(*) FROM Telecom GROUP BY CustomerID;
+-----+-----+
| CustomerID | COUNT(*) |
+-----+-----+
| C01       | 1        |
| C02       | 1        |
| C03       | 1        |
+-----+-----+
3 rows in set (0.01 sec)

mysql> SELECT PhoneNo, COUNT(*) FROM Telecom GROUP BY PhoneNo;
+-----+-----+
| PhoneNo | COUNT(*) |
+-----+-----+
| 9876543210 | 1        |
| 9876543211 | 1        |
| 9876543212 | 1        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT BillID, COUNT(*) FROM Telecom GROUP BY BillID;
mysql> SELECT BillID, COUNT(*) FROM Telecom GROUP BY BillID;
+-----+-----+
| BillID | COUNT(*) |
+-----+-----+
| B01    | 1        |
| B02    | 1        |
| B03    | 1        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT CustomerID, CustomerName, PhoneNo, BillID, BillDate, Amount FROM Telecom;
+-----+-----+-----+-----+-----+-----+
| CustomerID | CustomerName | PhoneNo | BillID | BillDate | Amount |
+-----+-----+-----+-----+-----+-----+
| C01       | Sonu        | 9876543210 | B01    | 2026-08-01 | 599.00 |
| C02       | Pranathi    | 9876543211 | B02    | 2026-08-02 | 799.00 |
| C03       | Roopa       | 9876543212 | B03    | 2026-08-03 | 499.00 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Result:

The SQL demonstrates candidate keys, primary key, uniqueness constraints, and the given functional dependencies before normalization.

8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

ANS:

Given Relation

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Primary Key

Assume each product has a unique PID.

Primary Key = PID

Functional Dependencies

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

$SupplierID \rightarrow SupplierName$

Therefore, there is a transitive dependency:

$PID \rightarrow SupplierID \rightarrow SupplierName$

Decompose

PRODUCT

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

PK = PID

SUPPLIER

SUPPLIER(SupplierID, SupplierName)

PK = SupplierID

SupplierID in PRODUCT is a foreign key.

BCNF

PRODUCT

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

PID is a key.

✓ BCNF

SUPPLIER

$SupplierID \rightarrow SupplierName$

SupplierID is a key.

✓ BCNF

Therefore:

Final Design = BCNF

Final Normalized Schema

PRODUCT

PID PK

PName

SupplierID FK

Category

Price

Warehouse

SUPPLIER

SupplierID PK

SupplierName

```

mysql> CREATE TABLE Product ( PID VARCHAR(10) PRIMARY KEY, PName VARCHAR(50) NOT NULL, SupplierID VARCHAR(10), Category VARCHAR(50), Price DECIMAL(10,2), Warehouse VARCHAR(50), FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID) );
Query OK, 0 rows affected (0.14 sec)

mysql> INSERT INTO Supplier VALUES ('S01', 'ABC Suppliers'), ('S02', 'XYZ Traders'), ('S03', 'Global Supplies');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Product VALUES ('P01', 'Laptop', 'S01', 'Electronics', 50000, 'W01'), ('P02', 'Mouse', 'S01', 'Accessories', 1000, 'W01'), ('P03', 'Keyboard', 'S02', 'Accessories', 2000, 'W02'), ('P04', 'Monitor', 'S03', 'Electronics', 15000, 'W02');
Query OK, 4 rows affected (0.02 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Product;
+----+-----+-----+-----+-----+-----+
| PID | PName | SupplierID | Category | Price | Warehouse |
+----+-----+-----+-----+-----+-----+
| P01 | Laptop | S01 | Electronics | 50000.00 | W01 |
| P02 | Mouse | S01 | Accessories | 1000.00 | W01 |
| P03 | Keyboard | S02 | Accessories | 2000.00 | W02 |
| P04 | Monitor | S03 | Electronics | 15000.00 | W02 |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Supplier;
+-----+-----+
| SupplierID | SupplierName |
+-----+-----+
| S01 | ABC Suppliers |
| S02 | XYZ Traders |
| S03 | Global Supplies |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT p.PID, p.PName, s.SupplierID, s.SupplierName, p.Category, p.Price, p.Warehouse FROM Product p JOIN Supplier s ON p.SupplierID = s.SupplierID;
+----+-----+-----+-----+-----+-----+
| PID | PName | SupplierID | SupplierName | Category | Price | Warehouse |
+----+-----+-----+-----+-----+-----+
| P01 | Laptop | S01 | ABC Suppliers | Electronics | 50000.00 | W01 |
| P02 | Mouse | S01 | ABC Suppliers | Accessories | 1000.00 | W01 |
| P03 | Keyboard | S02 | XYZ Traders | Accessories | 2000.00 | W02 |
| P04 | Monitor | S03 | Global Supplies | Electronics | 15000.00 | W02 |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

Result:

The original PRODUCT relation contains the transitive dependency $PID \rightarrow SupplierID \rightarrow SupplierName$. Therefore, supplier information is separated into a SUPPLIER table. The final PRODUCT and SUPPLIER relations eliminate redundancy and satisfy 3NF; under the stated dependencies, they also satisfy BCNF.

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

ANS:

Poorly Normalized Schema

```

SCHOOL_ERP(
    StudentID, StudentName, ClassID, ClassName,
    TeacherID, TeacherName, SubjectID, SubjectName, Marks
)

```

Functional Dependencies

```

StudentID → StudentName, ClassID
ClassID → ClassName, TeacherID
TeacherID → TeacherName
SubjectID → SubjectName
(StudentID, SubjectID) → Marks

```

Normalization

1NF: Atomic values → ✓

2NF: Remove partial dependencies:

```

STUDENT(StudentID, StudentName, ClassID)
SUBJECT(SubjectID, SubjectName)
ENROLLMENT(StudentID, SubjectID, Marks)

```

3NF: Remove transitive dependencies:

```

CLASS(ClassID, ClassName, TeacherID)
TEACHER(TeacherID, TeacherName)

```

Final BCNF Design

STUDENT(StudentID PK, StudentName, ClassID FK)

CLASS(ClassID PK, ClassName, TeacherID FK)

TEACHER(TeacherID PK, TeacherName)

SUBJECT(SubjectID PK, SubjectName)

ENROLLMENT(StudentID FK, SubjectID FK, Marks,

PK = StudentID + SubjectID)

```
mysql> CREATE DATABASE SchoolERP;
Query OK, 1 row affected (0.06 sec)

mysql> USE SchoolERP;
Database changed
mysql> CREATE TABLE Teacher ( TeacherID VARCHAR(10) PRIMARY KEY, TeacherName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Class ( ClassID VARCHAR(10) PRIMARY KEY, ClassName VARCHAR(50) NOT NULL, TeacherID VARCHAR(10), FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Student ( StudentID VARCHAR(10) PRIMARY KEY, StudentName VARCHAR(50) NOT NULL, ClassID VARCHAR(10), FOREIGN KEY (ClassID) REFERENCES Class(ClassID) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Subject ( SubjectID VARCHAR(10) PRIMARY KEY, SubjectName VARCHAR(50) NOT NULL );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Enrollment ( StudentID VARCHAR(10), SubjectID VARCHAR(10), Marks INT, PRIMARY KEY (StudentID, SubjectID), FOREIGN KEY (StudentID) REFERENCES Student(StudentID), FOREIGN KEY (SubjectID) REFERENCES Subject(SubjectID) );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO Teacher VALUES ('T01', 'Sonu'), ('T02', 'Pranathi');
Query OK, 2 rows affected (0.07 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Class VALUES ('C01', '10-A', 'T01'), ('C02', '10-B', 'T02');
Query OK, 2 rows affected (0.06 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Student VALUES ('S01', 'Ramesh', 'C01'), ('S02', 'Aishu', 'C01'), ('S03', 'Lehan', 'C02');
Query OK, 3 rows affected (0.07 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Subject VALUES ('SUB01', 'DBMS'), ('SUB02', 'Mathematics'), ('SUB03', 'Python');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Enrollment VALUES ('S01', 'SUB01', 85), ('S01', 'SUB02', 90), ('S02', 'SUB01', 88), ('S03', 'SUB03', 92);
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Teacher;
+-----+-----+
| TeacherID | TeacherName |
+-----+-----+
| T01       | Sonu        |
| T02       | Pranathi    |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Class;
+-----+-----+-----+
| ClassID | ClassName | TeacherID |
+-----+-----+-----+
| C01     | 10-A     | T01       |
| C02     | 10-B     | T02       |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | StudentName | ClassID |
+-----+-----+-----+
| S01       | Ramesh     | C01     |
| S02       | Aishu      | C01     |
| S03       | Lehan      | C02     |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Subject;
+-----+-----+
| SubjectID | SubjectName |
+-----+-----+
| SUB01     | DBMS        |
| SUB02     | Mathematics |
| SUB03     | Python      |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| StudentID | SubjectID | Marks |
+-----+-----+-----+
| S01       | SUB01     | 85    |
| S01       | SUB02     | 90    |
| S02       | SUB01     | 88    |
| S03       | SUB03     | 92    |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| StudentID | SubjectID | Marks |
+-----+-----+-----+
| S01       | SUB01     | 85    |
| S01       | SUB02     | 90    |
| S02       | SUB01     | 88    |
| S03       | SUB03     | 92    |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT s.StudentID, s.StudentName, c.ClassName, t.TeacherName, sub.SubjectName, e.Marks FROM Enrollment e JOIN Student s ON e.StudentID = s.StudentID
JOIN Class c ON s.ClassID = c.ClassID JOIN Teacher t ON c.TeacherID = t.TeacherID JOIN Subject sub ON e.SubjectID = sub.SubjectID;
+-----+-----+-----+-----+-----+-----+
| StudentID | StudentName | ClassName | TeacherName | SubjectName | Marks |
+-----+-----+-----+-----+-----+-----+
| S01       | Ramesh     | 10-A     | Sonu        | DBMS        | 85    |
| S01       | Ramesh     | 10-A     | Sonu        | Mathematics | 90    |
| S02       | Aishu      | 10-A     | Sonu        | DBMS        | 88    |
| S03       | Lehan      | 10-B     | Pranathi    | Python      | 92    |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.04 sec)
```

Result:

The poorly normalized School ERP data is decomposed into TEACHER, CLASS, STUDENT, SUBJECT, and ENROLLMENT. Primary and foreign keys maintain relationships, redundancy is reduced, and the final design satisfies the proposed BCNF structure.

10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

ANS:

Functional Dependencies

MovieID $\rightarrow$ MovieTitle

ActorID $\rightarrow$ ActorName

PlatformID $\rightarrow$ PlatformName

The main relationship is:

(MovieID, ActorID, PlatformID)

Join Dependencies

The three independent relationships are:

(MovieID, ActorID)

(MovieID, PlatformID)

(ActorID, PlatformID)

Therefore, the original relation can be decomposed into:

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

5NF Decomposition

MOVIE

MOVIE(MovieID, MovieTitle)

ACTOR

ACTOR(ActorID, ActorName)

PLATFORM

PLATFORM(PlatformID, PlatformName)

MOVIE_ACTOR

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM

ACTOR_PLATFORM(ActorID, PlatformID)

Normalization

Original Relation

↓

1NF

↓

2NF

↓

3NF

↓

BCNF

↓

5NF

↓

Remove Join Dependencies

↓

Final Relations

```
mysql> CREATE DATABASE MovieDB;USE MovieDB;
Query OK, 1 row affected (0.05 sec)

Database changed
mysql> CREATE TABLE Movie ( MovieID VARCHAR(10) PRIMARY KEY, MovieTitle VARCHAR(100) );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Actor ( ActorID VARCHAR(10) PRIMARY KEY, ActorName VARCHAR(50) );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Platform ( PlatformID VARCHAR(10) PRIMARY KEY, PlatformName VARCHAR(50) );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Movie_Actor ( MovieID VARCHAR(10), ActorID VARCHAR(10), PRIMARY KEY (MovieID, ActorID), FOREIGN KEY (MovieID) REFERENCES Movie(MovieID)
FOREIGN KEY (ActorID) REFERENCES Actor(ActorID) );
Query OK, 0 rows affected (0.18 sec)

mysql> CREATE TABLE Movie_Platform ( MovieID VARCHAR(10), PlatformID VARCHAR(10), PRIMARY KEY (MovieID, PlatformID), FOREIGN KEY (MovieID) REFERENCES Movie
MovieID), FOREIGN KEY (PlatformID) REFERENCES Platform(PlatformID) );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Actor_Platform ( ActorID VARCHAR(10), PlatformID VARCHAR(10), PRIMARY KEY (ActorID, PlatformID), FOREIGN KEY (ActorID) REFERENCES Actor
ActorID), FOREIGN KEY (PlatformID) REFERENCES Platform(PlatformID) );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Movie VALUES ('M01', 'Inception'), ('M02', 'Interstellar'), ('M03', 'Avatar');
Query OK, 3 rows affected (0.09 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Actor VALUES ('A01', 'Leonardo DiCaprio'), ('A02', 'Matthew McConaughey'), ('A03', 'Sam Worthington');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Platform VALUES ('P01', 'Netflix'), ('P02', 'Amazon Prime'), ('P03', 'Disney+');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Movie_Actor VALUES ('M01', 'A01'), ('M02', 'A02'), ('M03', 'A03');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Movie_Platform VALUES ('M01', 'P01'), ('M01', 'P02'), ('M02', 'P01'), ('M03', 'P03');
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Actor_Platform VALUES
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Actor_Platform VALUES
-> ('A01', 'P01'),
-> ('A01', 'P02'),
-> ('A02', 'P01'),
-> ('A03', 'P03');INSERT INTO Actor_Platform VALUES ('A01', 'P01'), ('A01', 'P02'), ('A02', 'P01'), ('A03', 'P03');
Query OK, 4 rows affected (0.08 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```

mysql> SELECT * FROM Movie;
+-----+-----+
| MovieID | MovieTitle |
+-----+-----+
| M01     | Inception  |
| M02     | Interstellar |
| M03     | Avatar     |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Actor;
+-----+-----+
| ActorID | ActorName  |
+-----+-----+
| A01     | Leonardo DiCaprio |
| A02     | Matthew McConaughey |
| A03     | Sam Worthington  |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Platform;
+-----+-----+
| PlatformID | PlatformName |
+-----+-----+
| P01        | Netflix      |
| P02        | Amazon Prime |
| P03        | Disney+     |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Movie_Actor;
+-----+-----+
| MovieID | ActorID |
+-----+-----+
| M01     | A01     |
| M02     | A02     |
| M03     | A03     |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Movie_Platform;
+-----+-----+
| MovieID | PlatformID |
+-----+-----+
| M01     | P01        |
| M02     | P01        |
+-----+-----+
mysql> SELECT * FROM Movie_Platform;
+-----+-----+
| MovieID | PlatformID |
+-----+-----+
| M01     | P01        |
| M02     | P01        |
| M01     | P02        |
| M03     | P03        |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Actor_Platform;
+-----+-----+
| ActorID | PlatformID |
+-----+-----+
| A01     | P01        |
| A02     | P01        |
| A01     | P02        |
| A03     | P03        |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT m.MovieID, m.MovieTitle, a.ActorID, a.ActorName, p.PlatformID, p.PlatformName FROM Movie m JOIN Movie_Actor ma ON m.MovieID = ma.MovieID JOIN
Actor a ON ma.ActorID = a.ActorID JOIN Movie_Platform mp ON m.MovieID = mp.MovieID JOIN Platform p ON mp.PlatformID = p.PlatformID JOIN Actor_Platform ap ON
a.ActorID = ap.ActorID AND p.PlatformID = ap.PlatformID;
+-----+-----+-----+-----+-----+-----+
| MovieID | MovieTitle | ActorID | ActorName  | PlatformID | PlatformName |
+-----+-----+-----+-----+-----+-----+
| M01     | Inception  | A01     | Leonardo DiCaprio | P01        | Netflix      |
| M01     | Inception  | A01     | Leonardo DiCaprio | P02        | Amazon Prime |
| M02     | Interstellar | A02     | Matthew McConaughey | P01        | Netflix      |
| M03     | Avatar     | A03     | Sam Worthington  | P03        | Disney+     |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

Result:

The final JOIN successfully reconstructs the movie, actor, and platform information from the decomposed tables, demonstrating the intended **lossless-join reconstruction** for the proposed 5NF design.